ReplayCapsule-RV: Event-Sufficient Hardware Failure Capsules for Replaying Embedded RISC-V Interrupt/MMIO Bugs

Anonymous Authors
Anonymous Institution

***Abstract*—Embedded firmware failures often depend on the order in which interrupts and memory-mapped I/O values reach a single-core processor. Full traces can capture that behavior, but they are costly to store and awkward to replay. ReplayCapsule-RV explores whether a single-hart RV32I target can replay safety failures from an event-sufficient capsule that records commit-indexed firmware-visible nondeterminism introduced by interrupts and MMIO. The artifact includes a model, an RV32I firmware interpreter, record-side RTL, PicoRV32 wrapper smokes, host-driven RTL replay rows, negative comparator tests, bounded local checks, scoped tiny iCE40 mapped rows, and generated baselines for six bug families. Compiler-backed firmware, full-core mapped overhead, true runtime overhead, and synthesizable replay-consume claims are not made unless generated by the corresponding scripts.**

I. Introduction

Embedded safety failures are often detected after the decisive nondeterministic event has already passed: a timer interrupt lands inside a critical region, a sensor read crosses a threshold, or a command register returns an unsafe value. A debug trace can preserve the whole execution, but the trace is usually much larger than the failure cause. A runtime monitor can flag the violation, but the monitor alone does not provide the replay-driving facts needed to reconstruct the same firmware-visible path.

ReplayCapsule-RV studies a narrower point in this design space. For a deterministic single-hart RV32I platform with the same firmware image and initial architectural state, the recorder captures the boundary events that can change the firmware-visible trace: MMIO read values, MMIO write observations, interrupt delivery/return facts, selected control-flow context, property-failure metadata, and optional diagnostic context. Events are ordered by commit index, so replay can consume each required event exactly once at the architectural boundary.

This paper makes three contributions:

- an event-sufficient replay model for interrupt/MMIO-induced RV32I failures under explicit single-hart assumptions;
- record-side RTL and replay-comparator infrastructure that share one event stream for property checking and capsule export;
- an artifact-backed evaluation that separates model, firmware-sim, RTL-smoke, formal, synthesis, and unavailable full-hardware gates.

TABLE I
Benchmark families and intended failure mechanisms. Generated coverage rows are in `results/processed/benchmark_coverage.csv`.

Family	Failure mechanism
Sensor threshold	high MMIO sensor value and deadline property
Interrupt race	interrupt delivery inside critical firmware window
MMIO ordering	unsafe command/output ordering at device boundary
Stack corruption	protected-store observation reaches checker
UART command	delayed unsafe command read drives actuator write
Watchdog timeout	missed heartbeat under long-running path

The contribution is intentionally scoped. ReplayCapsule-RV is not a replacement for RISC-V trace standards, post-silicon debug fabrics, snapshots, or full-system deterministic replay.

II. Motivation and Threat/Failure Model

The motivating failures are firmware bugs whose manifestation depends on external ordering rather than pure arithmetic determinism. The current benchmark suite covers sensor threshold handling, interrupt races, MMIO ordering, stack protection, UART command handling, and watchdog timeout behavior. Each family has a failing image and a fixed image; selected families also include a no-failure edge case. Table I summarizes the scope.

The threat model is a debugging and validation setting, not an attacker model. The recorder may observe firmware-visible events and property-checker outputs. Replay starts from the same image and initial architectural state. Unsupported nondeterminism is out of scope unless it is modeled as an event. In particular, multicore interleavings, DMA, caches/coherence effects, analog device state, and undefined C behavior are not covered by the current artifact.

III. Event-Sufficient Replay Model

The replay model is defined over architectural transitions. The original run and replay run share the same platform profile, firmware image, and initial architectural state. Between boundary events, the RV32I core semantics are deterministic. At a boundary, replay consumes the recorded event at the same commit index and validates replay-visible observations.

An event contains a commit index, a same-index order tag, a class, and a payload. Core classes are those required to drive replay in the scoped failures: MMIO read values, MMIO write

Generated source: paper/figures/replay_flow.svg

Fig. 1. Record/replay timeline with commit-indexed events. Source generated from scripts/make_figures.py.

Generated source: paper/figures/architecture_overview.svg

Fig. 2. ReplayCapsule hardware architecture. Source generated from scripts/make_figures.py.

observations, interrupt entry and exit, external inputs, selected stores or branch outcomes when they influence the property, checkpoint/hash values when used, and the property-failure signature. Diagnostic PC context is useful for debugging and comparator alignment, but the model does not claim global trace minimality.

Figure 1 shows the intended record/replay timeline. The recorder captures only the boundary facts needed to make the replay run take the same firmware-visible path through the target failure prefix. Overflow invalidates replay availability; it is a status bit, not a recoverable compression mode.

The proof sketch in formal/replay_sufficiency_theorem.md is by induction over commit index: equal initial state, deterministic ordinary steps, equal boundary-event payloads at nondeterministic steps, and deterministic property checking over the equal prefix.

IV. Hardware Architecture

ReplayCapsule-RV instruments the processor boundary rather than replacing the core. The PicoRV32 wrapper exposes memory, MMIO, interrupt, and commit-context signals to the record-side modules. The event tap observes candidate facts, the classifier selects replay-relevant and diagnostic classes, the slicer retains context around a property failure, the capsule buffer stores encoded packets, and the property checker freezes the capsule on a target violation.

The design keeps resource claims separate from functionality claims. Generic Yosys cell-count evidence is available for the baseline core, the recorder top, and the integrated wrapper. Scoped tiny iCE40 mapped rows are generated for a tiny baseline and recorder configuration; full-core FPGA or ASIC area/timing overhead claims are not made because comparable full-core mapped PASS reports are not present in the artifact.

V. Replay Engine and Verification

The replay comparator consumes an expected capsule and an observed replay trace. A passing comparison requires matching property metadata, the required event classes at the selected commit or cycle index, exact payload equality for replay-driving events, and no extra replay-driving events. Negative tests corrupt metadata, remove events, duplicate events, shift indices, swap order tags, and perturb payloads. The same comparator is used for model fixtures and exported RTL-smoke capsules.

Bounded local checks cover recorder and control contracts such as event classification, buffer behavior, property signatures, replay mismatch handling, register access, hash/signature update, and MMIO/interrupt logging. These checks support local RTL obligations; they are not a mechanized end-to-end processor theorem.

TABLE II
Evidence levels and gate status. Generated sources include results/processed/replay_experiments.csv, results/processed/rtl_capsule_exports.csv, and results/processed/full_rtl_replay.csv.

Level	Current role	Status meaning
Model	event-level executable replay fixtures	generated pass rows
Firmware-sim	RV32I interpreter replay fixtures	generated pass rows
RTL-smoke	wrapper runs and capsule export checks	generated pass rows
Full RTL	firmware-running record/replay suite	blocked rows
Mapped synthesis	FPGA or ASIC reports	unavailable rows

TABLE III
Replay success and corruption-test status. The detailed generated rows are in results/processed/replay_negative_tests.csv, results/processed/rtl_capsule_exports.csv, and results/processed/full_rtl_replay_negative.csv.

Evidence	Positive replay	Corruption behavior
Model	pass rows generated	negative rows reject corrupted capsules
Firmware-sim	pass rows generated	shares comparator checks
RTL-smoke	export self-checks generated	export-level corruptions rejected
Full RTL	45/45 evaluated rows pass	10 replay-critical corruptions reject, 2 ar

VI. Evaluation

The evaluation is generated by the artifact scripts rather than hand-entered into the paper. Model and firmware-sim replay rows are produced by scripts/run_replay_experiments.py. RTL-smoke export rows are produced by scripts/export_rtl_capsules.py. Baseline rows, ablation rows, scaling rows, mapped-flow status rows, and table sources are produced by the conference-evidence scripts.

A. Replay Success and Corruption Tests

The generated replay evidence currently supports model and firmware-sim replay for each benchmark family, RTL-smoke capsule export/comparator checks for failing and fixed images, and host-driven firmware-running RTL record/replay for all evaluated benchmark/variant/seed rows in results/processed/full_rtl_replay.csv. The RTL replay claim remains scoped to the current single-hart Verilator harness and verified fallback HEX images, not compiler-backed firmware or a synthesizable replay-consume datapath. Table II and paper/figures/table02_replay_evidence.md are the reviewer-facing summaries.

B. Baselines and Ablations

The baseline comparison includes full instruction, full commit, branch/control-flow, store/output, MMIO-only, interrupt-only, interrupt-plus-MMIO, snapshot-on-failure, core-only capsule, and property-aware capsule rows. Rows

Generated source: paper/figures/baseline_trace_sizes.svg

Fig. 3. Replay success and baseline comparison. Source generated from scripts/make_figures.py and the baseline CSVs.

Generated source: paper/figures/ablation_heatmap.svg

Fig. 4. Event-sufficiency ablation matrix. Source generated from scripts/make_figures.py and ablation CSVs.

that cannot replay by design are marked as failures; rows that lack a generated source remain unavailable. Figure 3 and results/processed/baseline_comparison.csv are the source-backed comparison.

The event-sufficiency matrix separates core replay-driving events from diagnostic context and unavailable evidence levels. It supports sufficiency within the generated scopes, not global minimality. Figure 4 and paper/figures/table_event_sufficiency.md summarize the ablation results.

C. Scaling and Hardware Cost

The scaling rows are synthetic model workloads used to separate fixed capsule overhead from longer executions. They are not firmware-running RTL evidence. The mapped synthesis table now contains scoped tiny iCE40 baseline and recorder place-and-route PASS rows, while the full PicoRV32 and wrapper mapped rows still fail placement and cannot support full-core overhead claims. Generic Yosys cell-count rows remain preliminary evidence and are shown separately from mapped rows in Table IV.

TABLE IV
HARDWARE OVERHEAD STATUS. GENERATED SOURCES ARE RESULTS/PROCESSED/SYNTHESIS.CSV, RESULTS/PROCESSED/SYNTHESIS_OVERHEAD.CSV, RESULTS/PROCESSED/MAPPED_SYNTHESIS.CSV, AND RESULTS/PROCESSED/MAPPED_OVERHEAD.CSV.

Flow	Current evidence	Claim al
Generic Yosys	measured cell-count rows	prelimina
Mapped FPGA	tiny baseline and recorder iCE40 PASS; full-core rows fail	scoped t
ASIC PnR	unavailable rows	no area

TABLE V
LIMITATIONS AND SCOPE. THE FULL GATE LEDGER IS DOCS/CONFERENCE_READINESS_DASHBOARD.MD.

Topic	Current scope
Processor	single-hart RV32I
Nondeterminism	commit-indexed interrupt/MMIO boundary events
Replay horizon	finite prefix through selected failure
Device model	replay-visible effects only
Hardware evidence	RTL-smoke, generic synthesis, and scoped tiny iCE40 mapping; no

VII. RELATED WORK

Processor trace standards and debug fabrics, including RISC-V trace work and vendor debug systems, aim to reconstruct program execution with rich control-flow or instruction-level visibility [1], [2]. ReplayCapsule-RV does not replace those mechanisms. It asks whether a scoped embedded failure can be replayed from the smaller set of firmware-visible boundary facts that drive the property failure.

Runtime verification and assertion monitors detect violations close to the hardware/software boundary [3]. ReplayCapsule-RV uses property checking as a capsule trigger, but its artifact focuses on replay-driving event capture and comparator evidence rather than only online detection.

Deterministic replay systems record nondeterministic inputs and ordering so an execution can be reconstructed [4], [5]. ReplayCapsule-RV borrows that replay discipline but narrows the target to single-hart embedded RV32I interrupt/MMIO failures and commit-indexed hardware capsules.

Snapshots and checkpoint systems capture enough state to resume or inspect execution [6]. The capsule design is complementary: it records boundary events and property context rather than claiming that a snapshot or full trace is unnecessary for every debugging task.

The open-source implementation builds on PicoRV32, Verilator, Icarus Verilog, Yosys, and bounded formal tooling [7], [8], [9], [10], [11]. These tools are part of the reproducibility story, not the scientific contribution.

VIII. LIMITATIONS

The scope is single-hart RV32I firmware with deterministic core semantics, the same program image, the same initial architectural state, and modeled external events. The current theorem excludes multicore interleavings, DMA unless modeled, cache/coherence effects unless modeled, analog nondeterminism, and claims about undefined C behavior.

MMIO side effects are modeled only at the replay-visible boundary. Interrupts are injected at an architectural boundary. Replay consumes every required event exactly once; missing, duplicate, reordered, or corrupted events must fail comparison. Capsule overflow invalidates replay availability.

The strongest generated evidence today is model, firmware-sim, RTL-smoke, host-driven full RTL replay, bounded local formal, generic synthesis, and scoped tiny iCE40 mapped evidence. Compiler-backed firmware, full-core mapped FPGA or ASIC overhead, and true runtime perturbation measurements are not claimed because the required generated reports do not exist in this artifact.

IX. CONCLUSION

ReplayCapsule-RV explores event-sufficient replay for embedded RV32I failures whose decisive nondeterminism comes through interrupts and memory-mapped I/O. The current artifact shows that the model, firmware-sim path, RTL-smoke exports, host-driven RTL replay rows, corruption checks, bounded local checks, scoped tiny mapped rows, and generated baselines can be reproduced without inventing missing hardware results. The remaining stronger evidence level is blocked until compiler-backed firmware, true runtime-overhead baselines, full-core mapped resource/timing reports, and a paper PDF build are generated.

REFERENCES

[1] RISC-V International, "RISC-V Processor Trace Specification," https://github.com/riscv-non-isa/riscv-trace-spec, accessed 2026-06-26.

[2] ——, "RISC-V External Debug Support," https://github.com/riscv/riscv-debug-spec, accessed 2026-06-26.

[3] M. Leucker and C. Schallhart, Eds., *Runtime Verification*. Springer, 2009.

[4] T. J. LeBlanc and J. M. Mellor-Crummey, "Debugging parallel programs with instant replay," in *IEEE Transactions on Computers*, 1987.

[5] G. W. Dunlap, S. T. King, S. Cinar, M. A. Basrai, and P. M. Chen, "Revirt: Enabling intrusion analysis through virtual-machine logging and replay," in *OSDI*, 2002.

[6] S. T. King, G. W. Dunlap, and P. M. Chen, "Debugging operating systems with time-traveling virtual machines," in *USENIX Annual Technical Conference*, 2005.

[7] C. Wolf, "PicoRV32: A Size-Optimized RISC-V CPU," https://github.com/YosysHQ/picorv32.

[8] Verilator Project, "Verilator," https://verilator.org/.

[9] Icarus Verilog Project, "Icarus Verilog," https://steveicarus.github.io/iverilog/.

[10] YosysHQ, "Yosys Open SYnthesis Suite," https://yosyshq.net/yosys/.

[11] ——, "SymbiYosys and Yosys SMTBMC," https://yosyshq.readthedocs.io/projects/sby/.